

5.4 Module Quiz

Date: 3/29/2026, 4:44:10 PM

Time Spent: 07:35

Score: 100%

Passing Score: 80%



In an AI-enhanced cloud environment, why might certain automated scaling or reconfiguration actions require human approval instead of being fully automated?

- ☐ Because changes that alter core network segments or security zones may need a human to validate potential side effects before deployment.
- ☐ Because any configuration update that improves performance should be postponed until a separate AI model verifies the expected cost savings.
- ☒ Because some high-impact changes must be manually authorized when usage exceeds preset limits, ensuring control over risk and cost. ✓ Correct
- ☐ Because adjustments to autoscaling policies should always be delayed until the AI can observe several weeks of additional traffic patterns.

Explanation

Because some high-impact changes must be manually authorized when usage exceeds preset limits, ensuring control over risk and cost is the correct answer. Once usage goes beyond predefined thresholds, human approval helps decide whether additional changes are justified given security, reliability, and budget.

Because changes that alter core network segments or security zones may need a human is incorrect. While such changes often do need review, the question emphasizes exceeding preset limits for scaling and reconfiguration, where overall risk and cost governance is the primary reason for human approval.

Because adjustments to autoscaling policies should always be delayed until the AI can observe several weeks of additional traffic patterns is incorrect. Historical data helps, but enforcing a fixed waiting period is not the main driver. The key issue is controlling significant changes that may have large operational or financial impact.

Because any configuration update that improves performance should be postponed is incorrect. Cost evaluation is useful, but human approval is mainly about overseeing high-impact changes beyond limits, not about requiring a secondary AI check before every performance improvement.

Related Content

 5.3.4 AI in Security Workflows

Question 2

✓ Correct

Which AI-enabled tool is BEST suited to interact with analysts in natural language to summarize log data, suggest next steps, and help walk through incident response procedures?

- ☐ Browser plug-ins
- ☐ Command-line interface plug-ins
- ☐ Integrated development environment plug-ins
- ☒ Chatbots ✓ Correct

Explanation

Chatbots is the correct answer. Chatbots are specifically designed to interact with users through natural language, making them well-suited to summarize logs, interpret alerts, and guide analysts step by step through incident response activities. They can answer questions, clarify procedures, and help standardize how tasks are carried out during investigations.

Integrated development environment plug-ins is incorrect. Integrated development environment plug-ins focus on assisting with code creation, review, and refactoring inside a coding environment. While they may include AI features, they are optimized for software development rather than conducting interactive, conversational guidance.

Browser plug-ins is incorrect. Browser plug-ins enhance web-based workflows, such as working with dashboards or online tools, but they are not primarily designed for back-and-forth natural language conversations about security tasks.

Command-line interface plug-ins is incorrect. Command-line interface plug-ins help users construct and understand commands at the terminal, often explaining syntax or generating scripts. They are useful for technical execution but are not primarily aimed at providing conversational guidance like a chatbot does.

Related Content



5.1.1 AI Tools in Security Operations

resources\questions\q_ai_tools_in_security_operations_04.question.xml

A security operations engineer is investigating why the organization's new AI-based email filter is suddenly allowing through a wave of sophisticated spam messages.

The messages look different from each other, with slight changes in wording, spacing, and character substitutions, yet they consistently bypass the model's classification while still looking legitimate to users.

The engineer suspects that attackers are using AI to repeatedly generate and refine these messages based on how the filter responds, gradually learning what will be accepted as "normal" email traffic.

Which technology is the attacker MOST likely using to systematically generate and test these evasive messages against the organization's AI defenses?

- ☐ Manual content review teams
- ☐ Rule-based spam filters
- ☒ Adversarial networks ✓ Correct
- ☐ Traditional signature databases for malware

Explanation

Adversarial networks is the correct answer. Adversarial networks can use a generator and a detector to iteratively create content that slips past defenses. Attackers can train one model to produce spam and another to mimic the email filter, refining the messages until they reliably evade the organization's AI.

Rule-based spam filters is incorrect. Rule-based spam filters rely on fixed patterns and keywords. They do not generate or adapt content against another AI system, so they cannot explain the automated, evolving spam in this scenario.

Traditional signature databases for malware is incorrect. Traditional signature databases store known malicious patterns but do not create or test new messages. They are used for detection, not for generating adaptive spam that learns from an AI filter's behavior.

Manual content review teams is incorrect. Manual review teams can adjust messages over time, but they cannot quickly generate and test large volumes of small variations. The high-speed, iterative evasion described points to automated adversarial techniques instead of human tuning.

Related Content



5.2.1 AI for Deception and Social Engineering

resources\questions\q_ai_for_deception_and_social_engineering_09.question.xml

An internal red team is assessing an organization that recently deployed several decoy systems designed to lure attackers. The red team uses an AI-driven tool that analyzes response times, service banners, data consistency, and traffic patterns from all discovered hosts.

Within minutes, the tool identifies a subset of machines whose replies are slightly too uniform, whose datasets lack realistic user activity, and whose service configurations do not match typical production baselines.

The tool flags these systems as low-value targets and shifts its focus to other hosts that appear more authentic.

Which AI-assisted technique is the tool MOST likely using to improve the efficiency of the attack?

- ☐ Automated vulnerability scanning
- ☐ Distributed denial of service (DDoS)
- ☒ Honeypots ✓ Correct
- ☐ Credential stuffing

Explanation

Honeypots is the correct answer. The behavior described matches AI-driven detection and avoidance of honeypots. The tool spots systems with uniform responses and unrealistic data, labels them as decoys, and targets more realistic hosts instead.

Automated vulnerability scanning is incorrect. Automated vulnerability scanning looks for flaws and misconfigurations across systems. The scenario instead focuses on identifying and avoiding deceptive systems, not just finding vulnerabilities everywhere.

Credential stuffing is incorrect. Credential stuffing involves mass login attempts with reused credentials. The scenario mentions analyzing host behavior and traffic patterns, not authentication attempts or password reuse.

Distributed denial of service (DDoS) is incorrect. A DDoS attack floods systems with traffic to disrupt availability. Here, the AI tool is classifying hosts and steering attacks away from decoys, not generating large volumes of traffic.

Related Content



5.2.4 AI for Automated Attacks

resources\questions\q_ai_for_automated_attacks_06.question.xml

In a CI/CD pipeline using AI-based software composition analysis (SCA), how should the findings typically influence deployment decisions?

- ☒ They should be used to gate releases by blocking builds that include high-risk dependencies until vulnerabilities or license issues are addressed. ✓ Correct
- ☐ They should primarily inform aesthetic code refactoring, such as reorganizing imports and renaming packages, while leaving functional behavior unchanged.
- ☐ They should only be reviewed when developers manually request a scan, so that normal integration and deployment workflows are not interrupted.
- ☐ They should be stored for post-deployment auditing while allowing all builds to proceed.

Explanation

They should be used to gate releases by blocking builds that include high-risk dependencies until vulnerabilities or license issues are addressed is the correct answer. In DevSecOps, SCA results become part of automated gates: if high-risk dependencies are found, the pipeline should fail or pause until the issues are resolved.

They should be stored for post-deployment auditing is incorrect. Relying on post-deployment audits allows known vulnerable components into production, which conflicts with the goal of catching problems early in CI/CD.

They should only be reviewed when developers manually request a scan is incorrect. SCA should run automatically and consistently, not just on manual request, so that dependency risk is checked every time code flows through the pipeline.

They should primarily inform aesthetic code refactoring is incorrect. SCA findings drive security and compliance actions like updating or removing libraries, not merely cosmetic refactoring of imports or package names.

Related Content



5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_03.question.xml

How can a low-code platform MOST effectively support a security analyst who needs a customized log analysis report?

- ☐ It automatically blocks all suspicious traffic without any configuration options, reporting, or ability to customize how logs are interpreted for different environments and threat models. It supplies prebuilt log analysis workflows and lets the analyst tweak parameters, but does not permit adding or editing any underlying code for custom parsing or enrichment.
- ☐ It includes a visual query builder that generates code automatically and supports limited script injection, but requires most complex transformations to be handled by manually written standalone tools.
- ☐ It requires the analyst to design every aspect of the report by writing complex SQL queries and Python code from the ground up. It offers a dashboard designer with drag-and-drop widgets, while any advanced filtering or correlation logic must be implemented through full-length scripts managed outside the platform.
- ☒ It provides a visual report builder with templates and allows the analyst to add a small script to adjust how specific log fields are filtered and displayed. ✓ Correct

Explanation

Providing a visual report builder with templates and allowing the analyst to add a small script to adjust how specific log fields are filtered and displayed is the correct answer. This aligns with low-code principles: the bulk of the reporting workflow is created through visual tools, while small, targeted code snippets give the analyst flexibility to customize filters, formatting, or field handling.

Offering a dashboard designer with drag-and-drop widgets is incorrect. This reduces the role of the low-code environment to display only. Because all meaningful customization is pushed to external, fully coded scripts, the platform is not truly low-code for the core log-analysis logic.

Supplying prebuilt log analysis workflows and lets the analyst tweak parameters is incorrect. This describes a configuration-only or no-code style system. While parameters can be adjusted, the inability to inject custom code means it lacks the extensibility expected in a low-code environment.

Including a visual query builder that generates code automatically and supports limited script injection is incorrect. Although it appears similar, this design offloads the majority of complex logic to separate, fully coded tools, so the low-code aspects apply only to simple cases, not the main customization needs of log analysis.

Related Content

 5.3.1 AI for Security Scripting and Content Summarization
resources\questions\q_ai_for_security_scripting_and_content_summarization_02.question.xml

A financial services company uses a CI/CD pipeline to deploy updates to its customer portal. During a recent incident, an automated scaling script opened more inbound ports than allowed by policy, exposing internal services.

Leadership now wants to tighten control over infrastructure changes while still maintaining fast deployments. The team decides to incorporate AI into their change-approval process.

Which approach should they implement?

- ☒ AI-assisted approvals ✓ Correct
- ☐ Manual approvals with AI-generated deployment scripts
- ☐ Post-deployment anomaly alerts without approval controls
- ☐ Fully autonomous AI-enforced changes

Explanation

AI-assisted approvals is the correct answer. This uses AI to analyze proposed changes and recommend approve, deny, or escalate, while humans make the final decision. It speeds and standardizes reviews without removing human oversight.

Fully autonomous AI-enforced changes is incorrect. Here the AI directly approves and applies changes without human review. This increases speed but creates a single point of failure and can push risky configurations into production unchecked.

Manual approvals with AI-generated deployment scripts is incorrect. AI is used only to generate scripts or templates, not to assess risk. Change decisions remain fully manual, so misconfigurations can still be missed under time pressure.

Post-deployment anomaly alerts without approval controls is incorrect. AI monitors for problems only after deployment. This may detect bad changes later but does nothing to prevent unsafe configurations from being approved and released in the first place.

Related Content



5.3.7 AI in DevSecOps

resources\questions\q_ai_in_devsecops_17.question.xml

What is a primary benefit of using AI-enabled platforms for threat modeling in complex infrastructures?

- ☐ They focus on aggregating known vulnerabilities into a single list without correlating them to data flows or trust boundaries.
- ☐ They help normalize diverse architectural inputs so that threats can be evaluated against a consistent risk scoring model.
- ☒ They can quickly process large configuration and telemetry datasets to produce tailored threat models and suggested mitigations. ✓ Correct
- ☐ They generate visual diagrams of the environment that show system relationships but do not associate those diagrams with attack patterns or controls.

Explanation

Quickly processing large configuration and telemetry datasets to produce tailored threat models and suggested mitigations is the correct answer. AI platforms ingest extensive environment data and use it to identify likely attack paths and gaps, then provide specific recommendations that match the actual infrastructure.

Helping normalize diverse architectural inputs is incorrect. Normalization and scoring help, but the main benefit is turning that data into concrete threat models and mitigation guidance, not just making scores comparable.

Focusing on aggregating known vulnerabilities into a single list is incorrect. Threat modeling must relate weaknesses to where data moves and where trust changes; simply listing vulnerabilities is more like basic vulnerability management.

Generating visual diagrams of the environment is incorrect. Diagrams are useful, but AI-enabled threat modeling is expected to tie those visuals to attacker techniques and recommended safeguards, making them actionable.

Related Content



5.1.6 AI Use Cases: Testing and Management

resources\questions\q_ai_use_cases_testing_and_management_04.question.xml

A security team receives a long incident report that includes detailed logs, timestamps, and analyst notes. They need a brief, high-level overview for executives that preserves the main events and outcomes.

Which AI use case is MOST appropriate?

☒ Summarization ✓ Correct

☐ Signature matching

☐ Vulnerability analysis

☐ Code quality and linting

Explanation

Summarization is the correct answer. Summarization reduces long, detailed reports into concise versions that keep key facts such as impact, timeline, and remediation, making them easier for executives to review quickly.

Code quality and linting is incorrect. Code quality and linting focus on reviewing source code for style, bugs, and maintainability issues, not on condensing narrative incident reports for stakeholders.

Vulnerability analysis is incorrect. Vulnerability analysis identifies and prioritizes system weaknesses. It helps assess risk but does not reduce lengthy reports into short, executive-ready overviews.

Signature matching is incorrect. Signature matching compares activity to known patterns of malicious behavior to detect threats; it does not transform documents into shorter, high-level summaries.

Related Content



5.1.6 AI Use Cases: Testing and Management

resources\questions\q_ai_use_cases_testing_and_management_06.question.xml

Which description BEST explains how AI-based data correlation helps attackers use breached credential dumps more effectively?

- ☐ It archives old breach data and deletes duplicate entries to reduce storage and simplify backup procedures for the attacker.
- ☐ It replaces all stolen credentials with randomly generated strings to test password policy strength in a simulated environment.
- ☒ It compares usernames and passwords across multiple dumps to infer likely password reuse on other platforms. ✓ Correct
- ☐ It encrypts each breach file separately so that defenders cannot easily analyze stolen credentials.

Explanation

Comparing usernames and passwords across multiple dumps to infer likely password reuse on other platforms is the correct answer. AI-based data correlation can quickly compare large sets of breached credentials, detect repeated usernames and password patterns, and suggest which passwords are most likely reused across different services. This transforms raw breach data into prioritized, actionable intelligence for further attacks.

Encrypting each breach file separately so that defenders cannot easily analyze stolen credentials is incorrect. Encrypting breach files may protect them from interception, but it does not correlate or analyze the contents. It is a storage and secrecy measure, not a process that extracts patterns or relationships from multiple data sources.

Archiving old breach data and deleting duplicate entries is incorrect. Archiving and deduplication address data management, not analytic value. While they might save space, they do not reveal cross-dump patterns or identify which credentials provide useful paths to additional accounts.

Replacing all stolen credentials with randomly generated strings is incorrect. Replacing real credentials with random strings destroys the meaningful relationships within the data. This prevents the very pattern recognition and cross-referencing that data correlation relies on to support targeted attacks.

Related Content

 5.2.3 AI for Reconnaissance and Data Correlation

Why should AI-enabled IDE plug-ins be carefully tested in a controlled environment before use in production workflows?

- ☐ They may integrate with other development tools and pipelines, so organizations must confirm that these interactions do not introduce unexpected behavior or security gaps.
- ☒ They might generate incorrect or insecure code and suggestions, so their behavior needs to be validated to reduce the risk of unintended changes. ✓ Correct
- ☐ They can influence how code is structured and refactored over time, so their impact on maintainability and long-term security needs to be evaluated in a safe setting.
- ☐ They can change how developers rely on automated recommendations, so teams need to verify that suggested patterns align with approved coding and security standards.

Explanation

They might generate incorrect or insecure code and suggestions, so their behavior needs to be validated to reduce the risk of unintended changes is the correct answer. This is accurate. Testing them in a controlled environment helps validate their behavior and prevent flawed suggestions from reaching production systems.

They can change how developers rely on automated recommendations is the incorrect answer. While it is important to ensure recommendations align with internal standards, this option focuses on developer behavior and style, not the primary risk of insecure or incorrect code that drives the need for controlled testing.

They may integrate with other development tools and pipelines is incorrect. Verifying integrations with other tools is useful but not the primary risk.

They can influence how code is structured and refactored over time is incorrect. Evaluating long-term effects on structure and maintainability matters, but the main motivation for controlled testing is to catch immediate security and correctness issues in generated code.

Related Content

 5.1.1 AI Tools in Security Operations

resources\questions\q_ai_tools_in_security_operations_01.question.xml

A threat intelligence analyst is reviewing unusual login attempts against a company's cloud portal. The attempts originate from many different IP addresses but share a common pattern: each attempt uses a slightly different username and password combination that closely resembles credentials leaked in several unrelated data breaches over the last five years.

Further investigation shows that an attacker is using an AI tool that rapidly aggregates old breach dumps, correlates reused passwords, and prioritizes likely account targets inside the company.

Which action by the attacker BEST demonstrates the use of AI for reconnaissance with automated data correlation in this scenario?

- ☒ Using AI to crawl public breach databases, cross-reference credential sets, and generate a prioritized list of likely valid usernames and passwords for the company's accounts. ✓ Correct
- ☐ Using AI-powered bots to flood the cloud portal with a massive volume of simultaneous connection requests, overwhelming its resources and making it unavailable to legitimate users.
- ☐ Using AI to automatically modify malware code in real time to evade antivirus signatures and behavior-based detection engines deployed on company endpoints.
- ☐ Using AI to generate a highly convincing spear-phishing email that mimics the writing style of the company's CEO and includes a link to a fake single sign-on page.

Explanation

Using AI to crawl public breach databases, cross-reference credential sets, and generate a prioritized list of likely valid usernames and passwords for the company's accounts is the correct answer. This is AI-enabled reconnaissance because it gathers information from many breach dumps, correlates reused credentials, and produces a prioritized target list. The focus is on discovering likely valid accounts before any direct exploitation.

Using AI to generate a highly convincing spear-phishing email that mimics the writing style of the company's CEO is incorrect. This is AI-assisted social engineering. The AI is used to craft a realistic phishing email to trick users, which is part of delivery and exploitation, not broad data gathering and correlation across multiple sources.

Using AI to automatically modify malware code in real time is incorrect. This describes AI-assisted malware evasion. Modifying code to bypass antivirus is about payload stealth during execution, not about collecting and correlating target information in the reconnaissance phase.

Using AI-powered bots to flood the cloud portal with a massive volume of simultaneous connection requests is incorrect. This is an AI-enhanced DDoS attack. AI helps overwhelm the service to disrupt availability, but it does not involve profiling users or correlating data about potential targets.

Related Content



5.2.3 AI for Reconnaissance and Data Correlation

resources\questions\q_ai_for_reconnaissance_and_data_correlation_03.question.xml

In a CI/CD process that uses AI for software composition analysis, what is the primary role of this capability?

- ☐ It prioritizes build performance by caching dependency downloads so that external libraries are retrieved faster during each pipeline run.
- ☐ It validates that declared dependencies match what is actually packaged in the build artifacts, helping detect missing or unexpected components.
- ☒ It examines third-party components and dependencies to identify vulnerabilities and compliance issues as code moves through the pipeline. ✓ Correct
- ☐ It tracks the versions of all frameworks and libraries so teams can schedule upgrades based solely on feature enhancements and release dates.

Explanation

Examining third-party components and dependencies to identify vulnerabilities and compliance issues as code moves through the pipeline is the correct answer. Software composition analysis focuses on the security and compliance posture of third-party components and dependencies as they pass through CI/CD.

Prioritizing build performance by caching dependency downloads is incorrect. Caching dependencies is a build-speed optimization, not the main security and compliance function of composition analysis.

Tracking down the versions of all frameworks and libraries is incorrect. Version tracking in SCA is aimed at highlighting vulnerable or noncompliant components, not planning feature-based upgrades.

Validating that declared dependencies match what is actually packaged in the build artifacts is incorrect. Detecting mismatches between declared and packaged dependencies can occur, but the primary goal of SCA is broader risk assessment, especially vulnerabilities and licensing issues.

Related Content

resources\questions\q_ai_in_devsecops_02.question.xml

How does AI most directly help malware evade traditional signature-based detection tools?

- ☒ By continuously reshaping the malware's code through automated obfuscation and minor variations while keeping its behavior similar. ✓ Correct
- ☐ By analyzing antivirus logs to schedule malware execution during low-activity periods when scanning is less frequent or more predictable.
- ☐ By learning which file formats and packaging methods are less likely to trigger alerts and wrapping the malware accordingly for delivery to targets.
- ☐ By automatically selecting from different prebuilt malware modules so that each infection uses a slightly different feature set and execution path.

Explanation

Continuously reshaping the malware's code through automated obfuscation and minor variations while keeping its behavior similar is the correct answer. AI can automatically generate many code-level variants of the same malware, changing structure and appearance while preserving function. This ongoing obfuscation makes static signatures that look for specific patterns far less reliable.

Automatically selecting from different prebuilt malware modules is incorrect. Varying prebuilt modules changes features and behavior, but the modules themselves can still match known signatures. AI-driven evasion is more focused on rewriting or transforming code so that underlying patterns no longer match existing signature rules.

Analyzing antivirus logs to schedule malware execution during low-activity periods is incorrect. Adjusting execution timing may reduce attention from analysts but does not change how signatures work. Signature-based tools will still flag known code patterns whenever scanning occurs, so timing alone does not directly defeat this type of detection.

Learning which file formats and packaging methods are less likely to trigger alerts is incorrect. Choosing less suspicious file formats or packing methods can help initial delivery but not the core signature problem. Once the malware is unpacked or runs in memory, its code is still examined, and unchanged patterns can still be matched by existing signatures.

Related Content



5.2.4 AI for Automated Attacks

resources\questions\q_ai_for_automated_attacks_02.question.xml

A financial services company deploys an AI-enabled monitoring system to protect its customer portal. Over several weeks, the system learns typical login patterns, including usual login times, geographic locations, device types, and transaction amounts for each user.

One evening, the system flags a series of logins where a single account suddenly initiates multiple high-value transactions from a foreign country, using a new device, at a time the user has never logged in before.

The security team wants the AI to automatically highlight these unusual behaviors compared to the user's established history so they can investigate possible account takeover.

Which AI use case is the system applying in this situation?

- ☐ Automated penetration testing
- ☐ Code quality and linting
- ☒ Anomaly detection ✓ Correct
- ☐ Signature matching

Explanation

Anomaly detection is the correct answer. The system is comparing current behavior—location, device, time, and transaction patterns—to what it has learned as normal for that user. Flagging deviations from a baseline of expected activity is the core of anomaly detection.

Signature matching is incorrect. Signature matching looks for known, predefined indicators such as specific malware patterns or rule-based signatures. The scenario instead highlights behavior that differs from normal, not matches to a fixed signature set.

Code quality and linting is incorrect. Code quality and linting focus on analyzing source code for style issues, logic clarity, and adherence to coding standards. The example involves monitoring user behavior and transaction activity, not reviewing or improving code.

Automated penetration testing is incorrect. Automated penetration testing involves simulating attacks against systems to discover weaknesses. In this case, the AI is observing live user behavior for irregularities, not launching test attacks against the environment.

Related Content

 5.1.3 AI Use Cases: Detection and Analysis

